arXiv:2303.10001v1 [cs.NI] 11 Mar 2023

Improving Data Transfer Efficiency for AIs in the DareFightingICE using gRPC

Chollakorn Nimpattanavong
Graduate School of Information Science and Engineering
Ritsumeikan University
Kusatsu, Japan
gr0608sp@ed.ritsumei.ac.jp

Ibrahim Khan
Graduate School of Information Science and Engineering
Ritsumeikan University
Kusatsu, Japan
gr0556vx@ed.ritsumei.ac.jp

Thai Van Nguyen
Graduate School of Information Science and Engineering
Ritsumeikan University
Kusatsu, Japan
gr0557fv@ed.ritsumei.ac.jp

Ruck Thawonmas
College of Information Science and Engineering
Ritsumeikan University
Kusatsu, Japan
ruck@is.ritsumei.ac.jp

Worawat Choensawat
School of Information Technology and Innovation
Bangkok University
Pathum Thani, Thailand
worawat.c@bu.ac.th

Kingkarn Sookhanaphibarn
School of Information Technology and Innovation
Bangkok University
Pathum Thani, Thailand
kingkarn.s@bu.ac.th

***Abstract*—This paper presents a new communication interface for the DareFightingICE platform, a Java-based fighting game focused on implementing AI for controlling a non-player character. The interface uses an open-source remote procedure call, gRPC to improve the efficiency of data transfer between the game and the AI, reducing the time spent on receiving information from the game server. This is important because the main challenge of implementing AI in a fighting game is the need for the AI to select an action to perform within a short response time. The DareFightingICE platform has been integrated with Py4J, allowing developers to create AIs using Python. However, Py4J is less efficient at handling large amounts of data, resulting in excessive latency. In contrast, gRPC is well-suited for transmitting large amounts of data. To evaluate the effectiveness of the new communication interface, we conducted an experiment comparing the latency of gRPC and Py4J, using a rule-based AI that sends a kick command regardless of the information received from the game server. The experiment results showed not only a 65% reduction in latency but also improved stability and eliminated missed frames compared to the current interface.**

***Index Terms*—Remote Procedure Call, gRPC, Producer-consumer synchronization, Fighting Game, DareFightingICE**

I. Introduction

FightingICE [1] is a fighting game platform[1] that is focused on implementing artificial intelligence (AI) for controlling a non-player character to fight against another computer-controlled character. In FightingICE, the game provides the AI with information about the current game state, allowing developers to create intelligent algorithms for the AI to use in order to decide on the best action to take within a short response time (16.66 ms in FightingICE). The challenge for developers is to create AI that is able to make quick and effective decisions based on the current game state in order to emerge victorious in the fighting arena. This platform is written in Java, and initially only supported AI development in Java.

Py4J [2] is a Python library that allows Python programs to access Java objects in a Java Virtual Machine (JVM) dynamically. It is a useful tool for integrating Python and Java code, and allows developers to call Java code from their Python programs and vice versa. Py4J is designed to be user-friendly and makes it easy to integrate Python and Java into a single application. The current version of the FightingICE platform has been integrated with Py4J implementation to provide a convenient way to combine the power of Python and Java, and has made it possible to use the strengths of Python programming to develop AIs for the FightingICE platform.

DareFightingICE [3] is an enhanced version[2] of the FightingICE platform that was proposed in 2022. This enhanced version includes improved sound design, allowing visually-impaired players to play the game, and providing a testing ground for AI algorithms that use sound as the sole input. However, the amount of time spent providing game information to the AI using Py4J is excessive, and sometimes exceeds the maximum response time, leaving no time for the AI to process the data. This is because Py4J relies heavily on sockets to communicate between Python and Java, which can add overhead and potentially limit its performance. In addition, Py4J is less efficient at handling large amounts of data or performing complex calculations compared to a native solution

[1]http://www.ice.ci.ritsumei.ac.jp/~ftgaic/

[2]https://tinyurl.com/DareFightingICE

arXiv:2303.10001v1 [cs.NI] 11 Mar 2023

使用gRPC提高DareFightingICE中人工智能的数据传输效率

Chollakorn Nimpattanavong立命馆大学信息科学与工程研究生院日本草津
g
r0608sp
@ed.ritsumei.ac.jp

Ibrahim Khan立命馆大学信息科学与工程研究生院日本草津
g
r
0556vx
@ed.ritsumei.ac.jp

Thai Van Nguyen立命馆大学信息科学与工程研究生院日本草津
g
r
0557fv
@ed.ritsumei.ac.jp

Ruck Thawonmas立命馆大学信息科学与工程学院日本草津
r
u
ck@is.ritsumei.ac.jp

Worawat Choensawat曼谷大学信息技术与创新学院泰国巴吞他尼
w
orawat.c@bu.ac.th

Kingkarn Sookhanaphibarn曼谷大学信息技术与创新学院泰国巴吞他尼
kingkarn.s@bu.ac.th

摘要——本文为DareFightingICE平台提出了一种新的通信接口。该平台是一款基于Java的格斗游戏，专注于实现控制非玩家角色的人工智能。该接口采用开源远程过程调用框架gRPC，以提高游戏与人工智能之间的数据传输效率，减少从游戏服务器接收信息所需的时间。这一点十分重要，因为在格斗游戏中实现人工智能的主要挑战在于，人工智能需要在很短的响应时间内选择要执行的动作。DareFightingICE平台已集成Py4J，使开发者能够使用Python创建人工智能。然而，Py4J处理大量数据的效率较低，会导致过高的延迟。相比之下，gRPC非常适合传输大量数据。为了评估新通信接口的有效性，我们开展了一项实验，比较gRPC与Py4J的延迟，所使用的基于规则的人工智能无论从游戏服务器接收到什么信息，都会发送踢击指令。实验结果表明，与现有接口相比，新接口不仅将延迟降低了65%，还提高了稳定性，并消除了丢失帧现象。

索引词——远程过程调用、gRPC、生产者-消费者同步、格斗游戏、DareFightingICE

I. 引言

FightingICE [1] 是一个格斗游戏平台[1]，专注于实现人工智能，以控制非玩家角色与另一个由计算机控制的角色对战。在 FightingICE 中，游戏会向人工智能提供当前游戏状态的信息，使开发者能够为人工智能设计智能算法，以便在很短的响应时间内（FightingICE 中为 16.66 毫秒）决定采取的最佳行动。面临这一挑战的开发者需要创建能够根据当前游戏状态快速做出有效决策的人工智能，从而在格斗竞技场中获胜。该平台使用 Java 编写，最初仅支持使用 Java 开发人工智能。

Py4J [2] 是一个 Python 库，允许 Python 程序动态访问 Java 虚拟机（JVM）中的 Java 对象。它是集成 Python 和 Java 代码的实用工具，允许开发者从 Python 程序中调用 Java 代码，反之亦然。Py4J 的设计注重用户友好性，可轻松将 Python 和 Java 集成到同一个应用程序中。当前版本的 FightingICE 平台已集成 Py4J 实现，为结合 Python 和 Java 的强大功能提供了便捷途径，并使开发者能够利用 Python 编程的优势，为 FightingICE 平台开发人工智能。

DareFightingICE [3] 是[2] 于2022年提出的 FightingICE 平台的增强版本。该增强版本改进了音效设计，使视障玩家也能游玩这款游戏，并为仅以声音作为输入的人工智能算法提供了测试平台。然而，使用 Py4J 向人工智能提供游戏信息耗时过长，有时甚至超过最大响应时间，导致人工智能没有时间处理数据。这是因为 Py4J 严重依赖套接字在 Python 和 Java 之间进行通信，这会增加开销，并可能限制其性能。此外，在处理大量数据或执行复杂计算时，Py4J 的效率不及原生解决方案

[1]http://www.ice.ci.ritsumei.ac.jp/~ftgaic/

[2]https://tinyurl.com/DareFightingICE

that is optimized for those specific tasks. This can be a problem for developers who want to create AI algorithms that require a significant amount of computational time or involve complex calculations in order to function effectively.

In this paper, we present a new communication interface for the DareFightingICE platform that uses an open-source remote procedure call, gRPC [4] instead of Py4J as shown in Figure 1. Our proposed interface is based on a producer-consumer approach [5], where the platform acts as the producer of data and the AI acts as the consumer. This approach helps to coordinate the generation and consumption of data more efficiently, and prevents potential problems such as data loss or corruption. Additionally, gRPC is capable of efficiently transmitting large amounts of data between the producer and consumer, making it an ideal choice for the DareFightingICE platform, where the AI must be able to quickly and effectively process large amounts of data in order to make decisions within the short response time.

II. Related Work

A. gRPC Remote Procedure Call

gRPC is an open-source remote procedure call framework that can run in any environment. It uses the HTTP/2 protocol [6] for transport, and Protocol Buffers [7] as the interface description language. gRPC enables client and server applications to communicate transparently, and simplifies the building of connected systems. One of the key features of gRPC is its ability to use a single HTTP/2 connection for bi-directional, full-duplex communication between client and server. This allows for the efficient exchange of large amounts of data and the ability to stream multiple messages in both directions. gRPC also supports a number of advanced features, such as authentication, flow control, blocking or non-blocking bindings, cancellation and timeouts. In addition, gRPC is designed to be highly performant. It uses a binary serialization format [8] that is compact and efficient, allowing for efficient transmission of data over the network.

B. Producer-Consumer Synchronization

Producer-consumer synchronization refers to the coordination of activities between producer and consumer threads [9] in a concurrent system. In a producer-consumer system, the producer thread is responsible for generating data, and the consumer thread is responsible for consuming the data. The key challenge in producer-consumer synchronization is to ensure that the producer and consumer threads coordinate their actions properly to avoid race conditions and other synchronization issues, while still allowing for efficient data transfer. This requires careful design and implementation of synchronization mechanisms, such as locks, semaphores, and monitors, to ensure that the producer and consumer threads can work together smoothly and efficiently.

C. Study on the performance of MCTS

In FightingICE, the sample AI was implemented using Monte Carlo Tree Search [10], called MCTSAI. Three different MCTSAIs with varying parameter settings were used in the study on [3]. These AIs, named MCTSAI165, MCTSAI115, and MCTSAI65, are based on a sample MCTSAI from FightingICE, but with the MCTS execution time set to 16.5 ms, 11.5 ms, and 6.5 ms, respectively. The execution time determines the time budget for the MCTS algorithm, so reducing the execution time can theoretically decrease the strength of the AI. The performance of each MCTSAI was evaluated by calculating the ratio of winning rounds to the total number of rounds played (300). The winning ratios for MCTSAI165, MCTSAI115, and MCTSAI65 were 0.96, 0.49, and 0.05, respectively. In conclusion, providing a longer computation time for the MCTS algorithm can lead to improved performance from the AI.

III. Proposed Communication Interface

This section covers the details of our proposed interface, which has two parts. The first part covers the exposed services on the DareFightingICE platform that can be accessed using gRPC. The second part covers the implementation of the DareFightingICE game server using a producer-consumer approach.

A. gRPC-accessible services

The gRPC server is integrated into the DareFightingICE platform, as shown in Figure 2. The server is typically served on port 50051 by default. The AI depicted in the figure serves as a guide for creating an AI that can effectively interact with this interface. There are three remote procedure call methods on the gRPC server side, allowing for easy communication and interaction with the platform. The AI must first register its information and request a server-streaming of the game state, as described as follows.

The *Initialize* method is a unary-RPC used to set up and configure an AI before the start of a game. This method requires a parameter of type *InitializeRequest*, which consists of a boolean value indicating the player number (i.e. true for player one and false for player two), the player's name, and a boolean value indicating whether the AI is blind or not. The response to this method is provided as an *InitializeResponse* object, which contains the player's unique identifier. This unique ID is required as a parameter for both the *Participate* and *Input* methods, which are used to control the AI's actions during the game.

The *Participate* method is a server-streaming RPC that allows AIs to register and receive game state information from the game server. The method takes *ParticipateRequest*, which consists of a player's unique identifier, as input. The output is a streaming of *PlayerGameData* object, which contains information about the game state, such as audio data. If the AI is not registered as blind AI, the *PlayerGameData* object will also include information about the current frame and screen data. This information can be used by the AI to make decisions and take actions in the game.

The *Input* method is a unary-RPC used for AIs to send their chosen actions to the game server. This method takes *PlayerAction* which consists of a player's unique identifier,

，后者针对这些特定任务进行了优化。对于希望开发需要大量计算时间或涉及复杂计算才能有效运行的人工智能算法的开发者而言，这可能是一个问题。

本文为DareFightingICE平台提出了一种新的通信接口，如图1所示，该接口使用开源远程过程调用框架 gRPC [4] 替代Py4J。我们提出的接口基于生产者-消费者方法 [5]，，其中平台充当数据的生产者，人工智能充当消费者。这种方法有助于更高效地协调数据的生成与消费，并防止数据丢失或数据损坏等潜在问题。此外，gRPC能够在生产者和消费者之间高效传输大量数据，因此非常适合DareFightingICE平台。在该平台中，人工智能必须能够快速、有效地处理大量数据，以便在较短的响应时间内做出决策。

II. 相关工作

A. gRPC远程过程调用

gRPC是一种可在任何环境中运行的开源远程过程调用框架。它使用HTTP/2协议[6]进行传输，并使用 Protocol Buffers [7] 作为接口描述语言。gRPC支持客户端和服务器应用程序进行透明通信，并简化互联系统的构建。gRPC的一个关键特性是能够使用单个HTTP/2连接，在客户端与服务器之间进行双向全双工通信。这使得双方能够高效交换大量数据，并在两个方向上以流式方式传输多条消息。gRPC还支持多种高级功能，例如身份验证、流量控制、阻塞或非阻塞绑定、取消和超时。此外，gRPC在设计上注重高性能。它采用紧凑、高效的二进制序列化格式 [8] ，从而实现数据在网络上的高效传输。

B. 生产者-消费者同步

生产者-消费者同步是指在并发系统中协调生产者线程和消费者线程[9] 的活动。在生产者-消费者系统中，生产者线程负责生成数据，消费者线程负责消费数据。生产者-消费者同步的关键挑战在于，确保生产者线程和消费者线程适当地协调各自的操作，以避免竞态条件和其他同步问题，同时仍能实现高效的数据传输。这需要精心设计和实现锁、信号量和管程等同步机制，以确保生产者线程和消费者线程能够顺畅、高效地协同工作。

C. 蒙特卡洛树搜索性能研究

在 FightingICE 中，示例人工智能采用蒙特卡洛树搜索 [10], 实现，名为 MCTSAI。研究使用了三种参数设置不同的 MCTSAI 在关于 [3] 的研究中。这些人工智能分别名为 MCTSAI165、MCTSAI115 和 MCTSAI65，均基于 FightingICE 的示例 MCTSAI，但将蒙特卡洛树搜索的执行时间分别设为 16.5 毫秒、11.5 毫秒和 6.5 毫秒。执行时间决定了蒙特卡洛树搜索算法的时间预算，因此，缩短执行时间理论上会削弱人工智能的实力。各 MCTSAI 的性能通过获胜回合数与总对战回合数（300）的比值进行评估。MCTSAI165、MCTSAI115 和 MCTSAI65 的胜率分别为 0.96、0.49 和 0.05。总之，为蒙特卡洛树搜索算法提供更长的计算时间可以提升人工智能的性能。

III. 提出的通信接口

本节详细介绍我们提出的接口，分为两个部分。第一部分介绍DareFightingICE平台上可通过gRPC访问的对外服务。第二部分介绍采用生产者-消费者方法实现DareFightingICE游戏服务器的方式。

A. 可通过gRPC访问的服务

gRPC服务器集成在DareFightingICE平台中，如图2所示。该服务器通常默认在端口50051上提供服务。图中所示的人工智能可作为创建能够与此接口有效交互的人工智能的参考。gRPC服务器端提供三种远程过程调用方法，便于与平台进行通信和交互。人工智能必须首先注册其信息，并请求以服务器流式传输方式获取游戏状态，具体如下所述。

Initialize 方法是一种一元RPC，用于在游戏开始前设置和配置人工智能。此方法需要一个类型为 *InitializeRequest*的参数，其中包含一个表示玩家编号的布尔值（即true表示玩家一，false表示玩家二）、玩家姓名，以及一个表示人工智能是否为盲人人工智能的布尔值。此方法以*InitializeResponse*对象的形式返回响应，其中包含玩家的唯一标识符。*Participate*和*Input* 方法都需要将此唯一标识符作为参数，这两种方法用于控制人工智能在游戏中的动作。

Participate 方法是一种服务器流式RPC，允许人工智能注册并从游戏服务器接收游戏状态信息。该方法以包含玩家唯一标识符的*ParticipateRequest*作为输入，输出则是由*PlayerGameData* 对象组成的数据流，其中包含音频数据等游戏状态信息。如果人工智能未注册为盲人工智能，*PlayerGameData* 对象还将包含当前帧信息和屏幕数据。人工智能可以利用这些信息在游戏中做出决策并采取行动。

Input 方法是一种一元RPC，供人工智能将其选定的动作发送到游戏服务器。此方法接受*PlayerAction* 作为输入，其中包含玩家的唯一标识符，

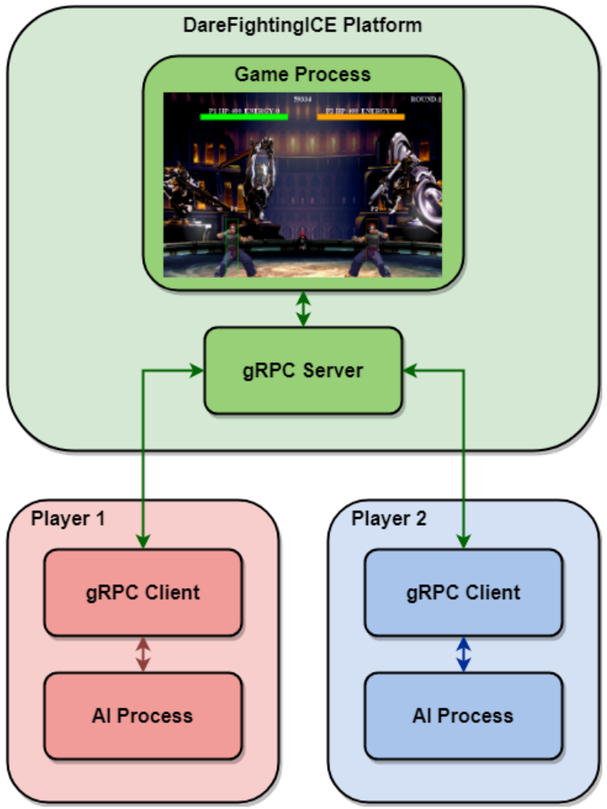

Fig. 1. Proposed system architecture

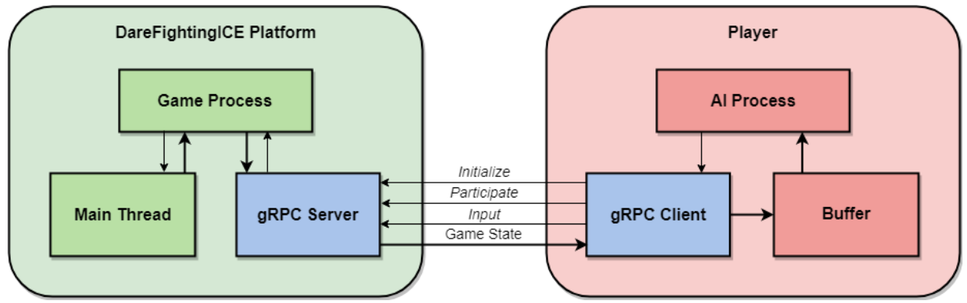

Fig. 2. The workflow of the gRPC-accessible services integrated into the DareFightingICE platform involves the flow of the game state from the main thread to the AI process, as indicated by the bold arrow. The flow of the player action from the AI process back to the main thread is indicated by the non-bold line.

along with a string representation of the selected action, as inputs. The game server will then use this information to update the game state. This allows the AI to participate in the game and make decisions based on its programmed strategy.

These methods can be used by AIs to send and receive information, and to participate in the game. The gRPC server and these methods provide a convenient and efficient way for AIs to interact with the DareFightingICE platform.

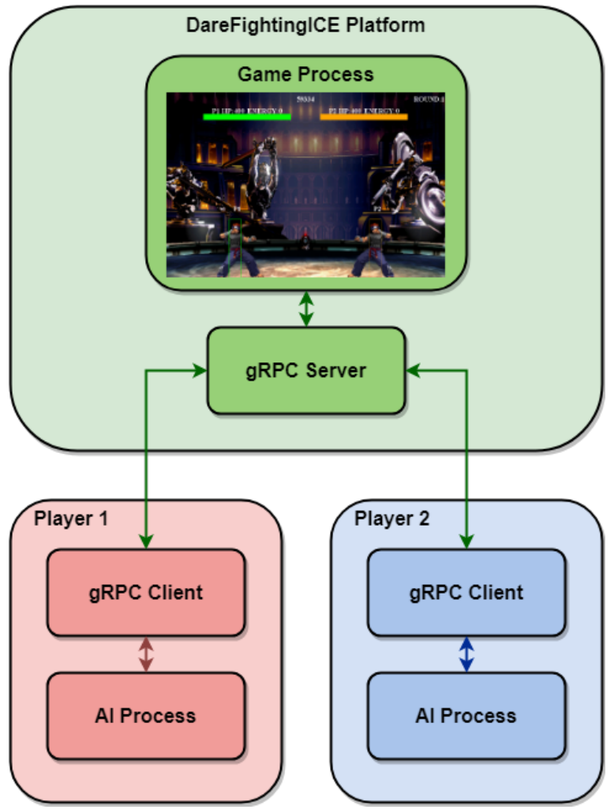

图1. 所提出的系统架构

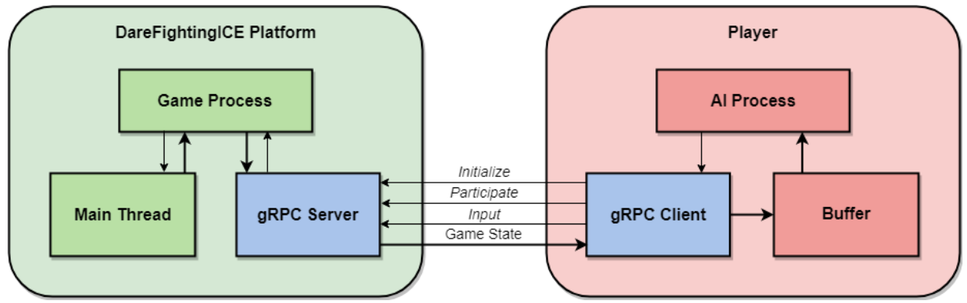

图2. 集成到DareFightingICE平台中的可通过gRPC访问的服务的工作流涉及游戏状态从主线程流向人工智能进程，如粗箭头所示。玩家动作从人工智能进程返回主线程的流向以非粗线表示。

以及所选动作的字符串表示。游戏服务器随后将使用这些信息来更新游戏状态。这使人工智能能够参与游戏，并根据其预编程策略做出决策。

人工智能可使用这些方法进行发送和接收

信息，并参与游戏。gRPC服务器和这些方法为人工智能与DareFightingICE平台交互提供了便捷高效的方式。

B. Game server with producer-consumer approach

In this proposed system, the DareFightingICE platform acts as a producer of game state information, while the AI acts as a consumer of this information. This system is based on the system architecture described in a previous study [1]. The DareFightingICE platform's processes consist of three main threads. The first is the main thread, which is responsible for all aspects of the game, including rendering the graphics and processing the game state. The other two threads are responsible for controlling the two AIs. These threads are called AI-Threads and are responsible for providing the AI with information about the current game state, processing the information, and updating the input received from the AI.

In the current system, only local processes are supported, so there is no way to implement an AI that can connect remotely to the DareFightingICE platform. To fix this, we modified the two AI-Threads to allow for the use of the producer-consumer approach and to provide the ability to connect to the platform remotely from a different process. In our modified system, the AI-Thread provides game state information to the AI and then halts until a response is received. Once the response is received, it is processed by the game server, and the responsible thread is resumed. This approach ensures efficient data transfer and avoids the possibility of the game server processing a command before the AI's action has been received.

IV. Evaluation

We implemented a rule-based AI that sends a kick command regardless of the information received from the game server. The purpose is to measure the amount of time the game server takes to provide information to the AI. We implemented two versions of this AI, one using Py4J and one using gRPC. We refer to the version implemented with Py4J as *Py4J_AI* and the version implemented with gRPC as *gRPC_AI*. To measure the amount of time spent, we started a timer when the information from the game server began transmitting to the AI, and ended the timer when the AI responded with the selected action to perform. The time was collected in nanoseconds and then divided it by one million to convert it to milliseconds. We did this to ensure precision, as milliseconds are critical in this scenario.

The PC used in the experiment had an Intel(R) Xeon(R) W-2123 @ 3.60GHz CPU, 16 GB DDR4 RAM, NVIDIA Quadro P400 graphics card, and ran on the Windows 11 Pro for Workstations operating system. The latency was measured on both the current and proposed interfaces, and the results are shown in Figure 3. In the figure, we have added two dashed lines. The first is a blue, vertical line at the x-axis value of 0, which shows the start of the game. The second is a red, horizontal line at the y-axis value of 16.66, which indicates the maximum response time for the AI. If the AI's response time exceeds this value, it means that the AI has no time left for processing anything.

The average latency using Py4J was 5.66 ms, while gRPC had an average latency of 1.99 ms. Our proposed interface significantly reduced the average latency by about 65% compared to the current one. During the experiment, we observed that the maximum latency using Py4J was 39.20 ms, while gRPC had a maximum latency of 6.18 ms which only occurred at the beginning of the game. In the later stages of the game, the latency for gRPC stabilized and remained consistently lower than that of Py4J. This indicates that our proposed interface is not only faster than the current one but also more stable.

TABLE I
Miss rate comparison

	Hit Frames	Miss Frames	Miss Rate
Py4J_AI	3,517	68	**1.90%**
gRPC_AI	3,585	0	**0.00%**

As mentioned earlier, exceeding the latency of 16.66 ms means that the AI will not be able to process anything, resulting in missed frames. Table I shows the miss rates for both *Py4J_AI* and *gRPC_AI*. In DareFightingICE, there are 3,600 frames per game. However, taking the 15 frame delay as described in [1] into account, the total number of frames that the AI is responsible for processing is 3,585 per game. The results show that *Py4J_AI* missed responding to the game server 68 times, which is 1.90% of all frames. In contrast, *gRPC_AI* was able to process everything without missing any frames. Therefore, the results show that our proposed communication interface can eliminate all miss rates in the current interface.

V. Discussions

The results of our experiments indicate that the proposed gRPC communication interface is significantly faster and more stable than the current Py4J interface. The gRPC interface reduced the average latency by about 65%, and eliminated all miss rates, which occurred in the Py4J interface. These improvements are important for the DareFightingICE platform, as a faster and more stable communication interface allows for better performance and responsiveness from the AIs.

One potential limitation of the proposed interface is the warm-up period that is required before it can be used efficiently. This warm-up period takes place during the pre-game phase and does not impact the performance of the game once it has begun. However, it may be worthwhile to investigate ways to reduce or eliminate this warm-up period in order to further improve the performance of the gRPC interface.

VI. Conclusions

In conclusion, we have proposed a new communication interface for the DareFightingICE platform that uses gRPC instead of Py4J. This interface is based on a producer-consumer approach, where the platform acts as the producer of data and the AI acts as the consumer. This approach helps to coordinate the generation and consumption of data more efficiently, and prevents potential problems such as data loss or corruption. Additionally, gRPC is capable of efficiently transmitting large

B. 采用生产者-消费者方法的游戏服务器

在所提出的系统中，DareFightingICE平台充当游戏状态信息的生产者，而人工智能则充当这些信息的消费者。该系统基于先前研究 [1]中描述的系统架构。DareFightingICE平台的进程由三个主要线程组成。第一个是主线程，负责游戏的各个方面，包括渲染图形和处理游戏状态。其余两个线程负责控制两个人工智能。这些线程称为人工智能线程，负责向人工智能提供当前游戏状态的信息、处理这些信息，以及更新从人工智能接收的输入。

当前系统仅支持本地进程，因此无法实现能够远程连接到DareFightingICE平台的人工智能。为解决这一问题，我们修改了两个人工智能线程，使其能够使用生产者-消费者方法，并支持从不同进程远程连接到该平台。在修改后的系统中，人工智能线程向人工智能提供游戏状态信息，随后暂停，直到收到响应。收到响应后，游戏服务器会对其进行处理，并恢复相应线程的运行。这种方法可确保高效的数据传输，并避免游戏服务器在收到人工智能的动作之前就处理指令。

IV. 评估

我们实现了一种基于规则的人工智能，无论从游戏服务器收到什么信息，它都会发送踢击指令。目的是测量游戏服务器向人工智能提供信息所需的时间。我们实现了该人工智能的两个版本，一个使用Py4J，另一个使用gRPC。我们将使用Py4J实现的版本称为*Py4J AI*，将使用gRPC实现的版本称为*gRPC AI*。为测量耗时，我们在游戏服务器开始向人工智能传输信息时启动计时器，并在人工智能返回所选的待执行动作时停止计时器。时间以纳秒为单位记录，然后除以一百万，将其转换为毫秒。这样做是为了确保精度，因为在此场景下，毫秒级的时间差至关重要。

实验所用的个人计算机配备主频为 3.60GHz 的英特尔至强 W-2123 中央处理器、16 GB DDR4 随机存取存储器和 NVIDIA Quadro P400 显卡，运行 Windows 11 专业工作站版操作系统。我们分别测量了现有接口和所提出接口的延迟，结果如图 3 所示。图中添加了两条虚线。第一条是位于 x 轴数值 0 处的蓝色竖线，表示游戏开始。第二条是位于 y 轴数值 16.66 处的红色横线，表示人工智能的最大响应时间。如果人工智能的响应时间超过该值，就意味着人工智能已没有剩余时间进行任何处理。

使用 Py4J 时的平均延迟为 5.66 毫秒，而使用 gRPC 时的平均延迟为 1.99 毫秒。我们提出的接口显著降低了平均延迟，与当前接口相比降幅约为 65%。在实验过程中，我们观察到，使用 Py4J 时的最大延迟为 39.20 毫秒，而使用 gRPC 时的最大延迟为 6.18 毫秒，且仅出现在游戏开始时。在游戏后期，gRPC 的延迟趋于稳定，并始终低于 Py4J 的延迟。这表明，我们提出的接口不仅比当前接口更快，而且更加稳定。

表 I 丢帧率比较

	命中帧	丢失帧	丢帧率
Py4J AI	3,517	68	**1.90%**
gRPC AI	3,585	0	**0.00%**

如前所述，当延迟超过 16.66 毫秒时，人工智能将无法进行任何处理，从而导致丢失帧。表 I 展示了 *Py4J AI* 和 *gRPC AI* 的漏失率。在 DareFightingICE 中，每局游戏共有 3,600 帧。然而，考虑到 [1]中所述的 15 帧延迟，人工智能每局实际需要处理的总帧数为 3,585 帧。结果表明，*Py4J AI* 有 68 次未能响应游戏服务器，占所有帧的 1.90%。相比之下，*gRPC AI* 能够完成全部处理，没有丢失任何帧。因此，结果表明，我们提出的通信接口能够将现有接口的漏失率降至零。

V. 讨论

实验结果表明，所提出的 gRPC 通信接口比当前的 Py4J 接口速度显著更快，稳定性也显著更高。gRPC 接口将平均延迟降低了约 65%，并完全消除了 Py4J 接口中出现的漏失率。这些改进对 DareFightingICE平台至关重要，因为更快、更稳定的通信接口能够提升人工智能的性能和响应能力。

所提出的接口有一个潜在局限，即在高效使用前需要经历预热期。该预热期发生在游戏开始前阶段，不会影响游戏开始后的性能。不过，为了进一步提升 gRPC 接口的性能，值得研究如何缩短或消除这一预热期。

VI. 结论

总之，我们为DareFightingICE平台提出了一种新的通信接口，使用gRPC代替Py4J。该接口基于生产者-消费者方法，其中平台充当数据的生产者，人工智能充当消费者。这种方法有助于更高效地协调数据的生成与消费，并防止数据丢失或数据损坏等潜在问题。此外，gRPC能够高效传输大量

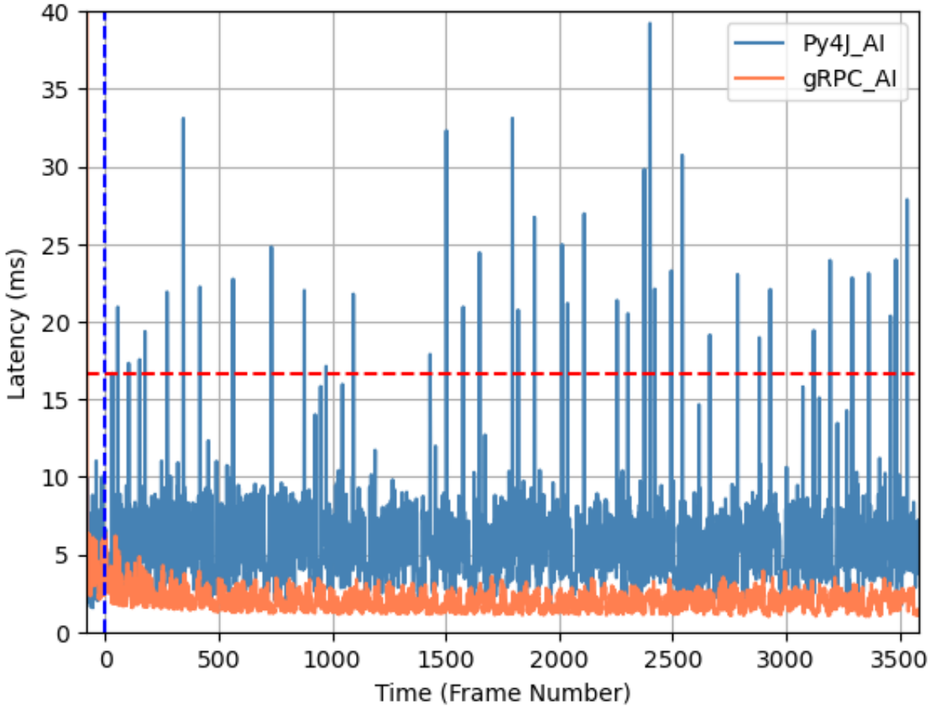

Fig. 3. Latency comparison

amounts of data between the producer and consumer, making it an ideal choice for the DareFightingICE platform.

We have also implemented two versions of AI to measure the amount of time the game server takes to provide information to the AI. Our experiments have shown that the proposed interface significantly reduces the average latency by about 65% compared to the current interface, and also eliminates all miss rates. Therefore, our proposed communication interface is a valuable addition to the DareFightingICE platform, providing improved performance and stability.

ACKNOWLEDGEMENT

The first three authors would like to express their gratitude to the Japanese Government for providing them with MEXT scholarships during their graduate studies at the Intelligent Computer Entertainment Laboratory, Graduate School of Information Science and Engineering, Ritsumeikan University, under the guidance and supervision of the fourth author.

REFERENCES

[1] F. Lu, K. Yamamoto, L. H. Nomura, S. Mizuno, Y. Lee, and R. Thawonmas, "Fighting Game Artificial Intelligence Competition Platform," in *Proceedings of the 2nd IEEE Global Conference on Consumer Electronics (GCCE)*, pp. 320-323, October 2013.

[2] "Py4J: Enabling Python programs to call Java code.," *https://www.py4j.org*, accessed December 26, 2022.

[3] I. Khan, T. V. Nguyen, X. Dai, R. Thawonmas, "DareFightingICE Competition: A Fighting Game Sound Design and AI Competition," in *2022 IEEE Conference on Games (CoG)*, pp. 478-485, August 2022.

[4] "Introduction to gRPC," *https://grpc.io/docs/what-is-grpc/introduction/*, accessed December 26, 2022.

[5] M. Manivannan, A. Negi and P. Stenström, "Efficient Forwarding of Producer-Consumer Data in Task-Based Programs," in *42nd International Conference on Parallel Processing*, pp. 517-522, October 2013.

[6] H. de Saxcé, I. Oprescu and Y. Chen, "Is HTTP/2 really faster than HTTP/1.1?," in *2015 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 293-299, May 2015.

[7] "Overview - Protocol Buffers," *https://developers.google.com/protocol-buffers/docs/overview*, accessed December 26, 2022.

[8] K. Maeda, "Performance evaluation of object serialization libraries in XML, JSON and binary formats," in *Second International Conference on Digital Information and Communication Technology and it's Applications (DICTAP)*, pp. 177-182, May 2012.

[9] A. M. Abhishek Sai, D. Reddy, P. Raghavendra, G. Y. Kiran and R. V. R, "Producer-Consumer problem using Thread pool," in *3rd International Conference for Emerging Technology (INCET)*, pp. 1-5, May 2022.

[10] M. Ishihara, T. Miyazaki, C. Y. Chu, T. Harada, and R. Thawonmas, "Applying and improving Monte-Carlo Tree Search in a fighting game AI," in *Proceedings of the 13th international conference on advances in computer entertainment technology*, pp. 1-6, November 2016.

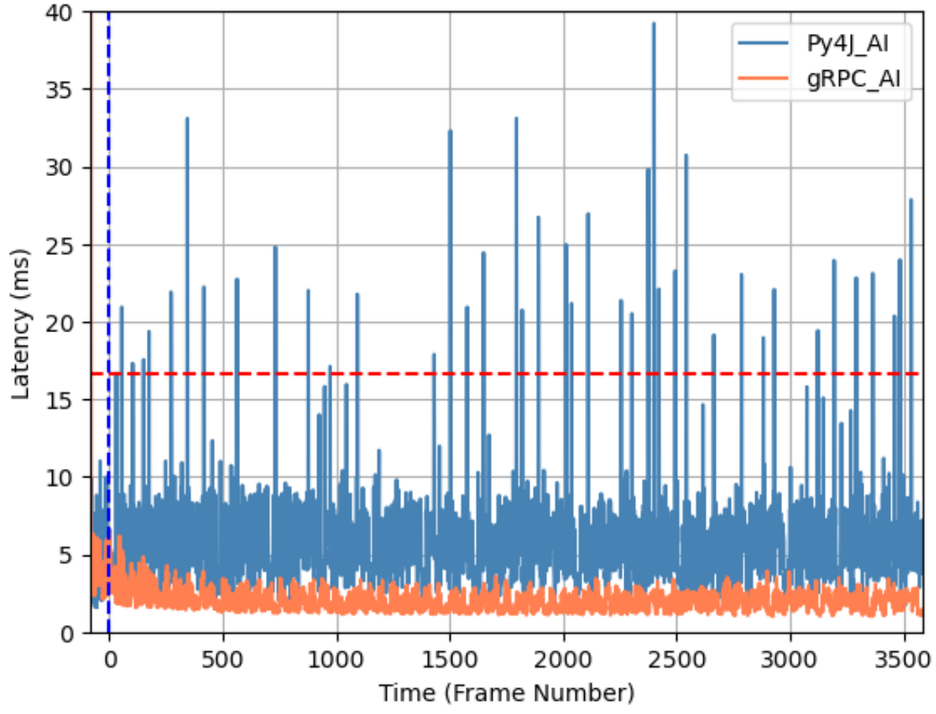

图3. 延迟比较

数据，在生产者与消费者之间实现传输，因此是DareFightingICE平台的理想选择。

我们还实现了两个版本的人工智能，以测量游戏服务器向人工智能提供信息所需的时间。实验表明，与现有接口相比，所提出的接口将平均延迟显著降低了约65%，同时将所有漏失率降至零。因此，我们提出的通信接口为DareFightingICE平台提供了有价值的补充，提升了其性能和稳定性。

致谢

前三位作者谨向日本政府致以谢意，感谢其在他们于立命馆大学信息科学与工程研究生院智能计算机娱乐实验室攻读研究生期间提供日本文部科学省奖学金。在此期间，他们接受了第四位作者的指导与监督。

参考文献

[1] F. Lu、K. Yamamoto、L. H. Nomura、S. Mizuno、Y. Lee 和 R. Thawonmas，"格斗游戏人工智能竞赛平台"，载于第二届*IEEE*全球消费电子大会（*GCCE*）论文集，第320-323页，2013年10月。

[2] "Py4J：使 Python 程序能够调用 Java 代码"，*https://www.py4j.org*，访问日期：2022年12月26日。[3] I. Khan、T. V. Nguyen、X. Dai、R. Thawonmas，"DareFightingICE竞赛：格斗游戏声音设计与人工智能竞赛"，载于*2022*年*IEEE*游戏大会（*CoG*），第478-485页，2022年8月。[4] "gRPC 简介"，*https://grpc.io/docs/what-is-grpc/introduction/*，访问日期：2022年12月26日。[5] M. Manivannan、A. Negi 和 P. Stenstr¨om，"基于任务的程序中生产者—消费者数据的高效转发"，载于第*42*届国际并行处理会议，第517-522页，2013年10月。[6] H. de Saxc´e、I. Oprescu 和 Y. Chen，"HTTP/2 真的比 HTTP/1.1 更快吗？"，载于*2015*年*IEEE*计算机通信大会研讨会（*INFOCOM WKSHPS*），第293-299页，2015年5月。[7] "概述 - Protocol Buffers"，*https://developers.google.com/protocol- buffers/docs/overview*，访问日期：2022年12月26日。[8] K. Maeda，"XML、JSON 和二进制格式对象序列化库的性能评估"，载于第二届数字信息与通信技术及其应用国际会议（*DICTAP*），第177-182页，2012年5月。[9] A. M. Abhishek Sai、D. Reddy、P. Raghavendra、G. Y. Kiran 和 R. V. R，"使用线程池解决生产者—消费者问题"，载于第三届新兴技术国际会议（*INCET*），第1-5页，2022年5月。[10] M. Ishihara、T. Miyazaki、C. Y. Chu、T. Harada 和 R. Thawonmas，"蒙特卡洛树搜索在格斗游戏人工智能中的应用与改进"，载于第*13*届计算机娱乐技术进展国际会议论文集，第1-6页，2016年11月。